

PROJECT REPORT

CAMPUS BUS PASS MANAGEMENT SYSTEM

A Java-Based Campus Transportation Management Application

Submitted for	VITyarthi Project Review
Student Name	Paras Dwivedi
Registration No.	25BAI10621
Technology	Java
Project Type	Console / Core Java Application
Main Package	com.campusbuspass
University	VIT Bhopal University

Abstract

The Campus Bus Pass Management System is a Java-based console application for basic campus transportation pass management. It provides a menu-driven workflow for adding student details, adding bus routes, generating monthly bus passes, and verifying pass validity. A generated pass is associated with a student registration number and a selected route and receives a unique pass ID. The implementation is separated into model, DAO, service, database, and utility packages.

The current implementation validates student and route existence before issuing a pass, uses the fee stored for the selected route, assigns validity dates, and handles invalid console input. The included execution screenshots demonstrate the main menu, student creation, route creation, pass generation, and pass verification.

1. Introduction

Campus transportation requires a simple way to maintain student information, route information, and bus-pass validity. Manual handling can make these operations less organized. This project applies Java programming and object-oriented design to implement a small console-based campus bus pass management application.

The application uses a numbered menu. The available operations are Add student, Add route, Generate pass, Verify pass, and Exit.

2. Problem Statement

A campus bus service needs a systematic method to maintain student details, maintain route details, issue bus passes, and verify whether a supplied pass is valid. The project addresses this need by linking a student to a selected route through a generated pass ID and providing a verification operation.

3. Objectives

- Maintain basic student registration information.
- Maintain campus bus route information including source, destination, and fee.
- Generate a unique bus pass ID for a valid student and route.
- Assign validity dates to a generated monthly pass.
- Verify a supplied pass ID.
- Validate student and route existence before issuing a pass.
- Use the actual fee configured for the selected route.
- Apply modular and object-oriented Java programming concepts.

4. Scope of the Project

- Student creation and lookup by registration number.
- Route creation and lookup by route ID.
- Monthly bus pass generation.
- Unique pass ID generation.
- Pass verification.
- Console-based interaction.
- Persistence through the project's local data/storage classes.

The current menu exposes the four core operations listed above plus Exit. The project also contains dedicated classes for pass, route, student, verification-log, storage, and input handling.

5. Requirements

5.1 Functional Requirements

ID	Requirement	Implementation
FR-01	Add student	Store student details and reject duplicate registration numbers.
FR-02	Add route	Create a route with an automatically assigned ID and store its fee.
FR-03	Generate pass	Validate student and route, create a unique pass ID, and save the pass.
FR-04	Verify pass	Accept a pass ID and return the verification result.
FR-05	Route fee	Use the actual fee stored for the selected route.
FR-06	Date validation	Reject an expiry date earlier than the start date.

5.2 Non-Functional Requirements

- Maintainability: responsibilities are separated across packages and classes.
- Readability: descriptive class and method names are used.
- Usability: a simple numbered console menu guides the operator.
- Validation: invalid student or route references are rejected during pass generation.
- Extensibility: the layered structure allows future interface or storage improvements.

6. System Architecture

The project follows a layered structure. Main handles the menu and coordinates user actions. Model classes represent application data. DAO classes provide data operations. Service classes contain business logic. DB and Schema provide the data/storage abstraction, while Input centralizes console input handling.

Layer / Package	Responsibility
Main	Entry point, menu, and user interaction coordination
model	Student, Route, Pass, and VerificationLog entities
dao	StudentDAO, RouteDAO, PassDAO, and VerificationLogDAO data operations
service	PassIdGenerator, PassService, and VerificationService business logic
db	DB and Schema data/storage setup
util	Input.java for console input

7. Final Repository Structure

```
Campus-Bus-Pass-Management-System/  
├── .vscode/  
├── src/  
│   ├── com/  
│   │   └── campusbuspass/  
└──
```

```

├── Main.java
├── dao/
│   ├── PassDAO.java
│   ├── RouteDAO.java
│   ├── StudentDAO.java
│   └── VerificationLogDAO.java
├── db/
│   ├── DB.java
│   └── Schema.java
├── model/
│   ├── Pass.java
│   ├── Route.java
│   ├── Student.java
│   └── VerificationLog.java
├── service/
│   ├── PassIdGenerator.java
│   ├── PassService.java
│   └── VerificationService.java
├── util/
│   └── Input.java
├── screenshots/
│   ├── main-menu.png
│   ├── add-student.png
│   ├── add-route.png
│   ├── generate-pass.png
│   └── verify-pass.png
├── report/
│   └── Campus_Bus_Pass_Management_System_Final_Report.docx
├── TERMINAL OUTPUT/
│   └── terminal output.pdf
├── .gitignore
├── README.md
└── campus_bus_passes.dat (local ignored data file)

```

The compiled out/ directory is generated locally for execution and is excluded from source control through .gitignore. The local campus_bus_passes.dat file is also ignored so runtime data changes are not committed to the repository.

8. Module Description

8.1 Student Module

Student and StudentDAO represent and manage student information. StudentDAO supports adding, finding, listing, updating, and deleting students, with registration-number lookup performed case-insensitively and duplicate registration numbers rejected.

8.2 Route Module

Route and RouteDAO represent and manage bus routes. A route contains an ID, route name, source, destination, distance, and fee. RouteDAO assigns an ID when required and provides lookup and CRUD operations.

8.3 Pass Module

PassService generates a monthly pass after checking the date range, student existence, and route existence. It reads the selected route's actual fee, generates a unique pass ID, creates a Pass object, and stores it through PassDAO.

8.4 Verification Module

VerificationService accepts a pass ID and checks its validity/status. The project also contains VerificationLog and VerificationLogDAO classes for verification-log support.

8.5 Input Utility Module

Input.java centralizes console input operations so that menu and data-entry code does not repeat scanner handling.

9. Core Algorithms

9.1 Pass Generation Algorithm

1. Start pass generation.
2. Read the student's registration number and route ID.
3. Set the pass validity dates.
4. Find the student using StudentDAO.
5. If the student does not exist, display an error and stop.
6. Find the route using RouteDAO.
7. If the route does not exist, display an error and stop.
8. Read the actual fee from the selected Route object.
9. Generate a unique pass ID using PassIdGenerator.
10. Create the Pass object with student, route, fee, dates, and active status.
11. Save the pass using PassDAO.
12. Display the generated pass ID.

9.2 Pass Verification Algorithm

13. Read the pass ID from the user.
14. Search for the pass.
15. Check pass validity/status and applicable expiry information.
16. Return the verification result.
17. Record verification information through the verification-log mechanism.

10. Implementation Details

The application is written in Java under the com.campusbuspass package. The implementation uses classes and objects, encapsulation, constructors, methods, lists, exception-based validation, LocalDate for validity dates, and separation of business logic from data-access operations.

Main provides the menu. PassService contains pass-issuance business rules. StudentDAO and RouteDAO provide lookup and storage operations. Pass represents an issued pass, while PassIdGenerator creates unique pass identifiers.

11. Validation and Error Handling

- Duplicate student registration numbers are rejected.
- A pass cannot be issued for a registration number that does not exist.
- A pass cannot be issued for a route ID that does not exist.
- An expiry date earlier than the validity start date is rejected.
- Invalid menu input is handled by the console input utility.

12. Testing

Testing was performed through the console interface using the main application workflow. The execution evidence supplied with the project demonstrates successful student creation, route creation, pass generation, pass verification, and program exit.

Test ID	Test Case	Expected / Observed Result	Status
TC-01	Start application	Menu displayed	Passed
TC-02	Add student	Student added	Passed
TC-03	Add route	Route added with generated ID	Passed
TC-04	Generate pass	Pass generated with unique ID and validity dates	Passed
TC-05	Verify pass	Valid pass returns successful verification	Passed

13. Execution Screenshots

The following screenshots are included in the repository under /screenshots. They show the working console application and the five main user-facing operations.

```
PS C:\Users\dwive\Desktop\vit java project> java -cp .\out com.campusbuspass.Main
== CAMPUS BUS PASS SYSTEM ==
1. Add student
2. Add route
3. Generate pass
4. Verify pass
0. Exit
Choose: |
```

Figure 1. Main menu

```
Choose: 1
Registration number: XPC
Name: BHUMIKA
Category: 1
Phone: 73
Student added.
```

Figure 2. Student creation

```
Choose: 2
Route name: COLLEGE
Source: BUS
Destination: DELHI
Fee: 10000
Route added with ID: 7
```

Figure 3. Route creation

```
Choose: 3
Registration number: XPC
Route ID: 7

PASS GENERATED SUCCESSFULLY

Pass ID: BP-2026-00006
Registration No: XPC
Route ID: 7
Valid From: 2026-09-13
Valid To: 2026-10-13
Status: ACTIVE
```

Figure 4. Pass generation

```
Choose: 4
Pass ID: BP-2026-00006
VALID: OK
```

14. Expected User Workflow

18. Launch the application.
19. Select 1 to add a student.
20. Select 2 to add a bus route and enter its fee.
21. Select 3 to generate a pass using an existing student and route.
22. Receive a unique pass ID and validity dates.
23. Select 4 and enter the pass ID to verify it.
24. Select 0 to exit.

15. Advantages

- Simple menu-driven console interface.
- Clear separation of concerns through layered packages.
- Unique pass identification.
- Student and route validation before pass issuance.
- Actual route fee is used for the pass.
- Verification is available directly from the main menu.
- Structure is suitable for future extension.

16. Limitations and Future Enhancements

The following are future enhancements and are not claimed as current menu features:

- Add a graphical user interface using Java Swing or JavaFX.
- Add dedicated screens for listing, updating, and deleting students and routes.
- Add richer pass details and printable pass output.
- Use stronger database storage such as SQLite or MySQL if required.
- Add automated unit tests for service and DAO classes.
- Add role-based access for transport administrators.
- Add reporting for active, expired, and blocked passes.

17. Learning Outcomes

The project demonstrates practical application of Java concepts including classes and objects, encapsulation, packages, modular design, collections, exception handling, date handling, and separation of business logic from data-access operations. It also demonstrates a structured development approach from problem identification through implementation and testing.

18. Conclusion

The Campus Bus Pass Management System provides a focused solution for basic campus bus-pass management. Its menu-driven workflow covers student entry, route entry, pass generation, and verification. The layered Java structure improves maintainability, while validation of student and route references improves the correctness of pass generation. The included execution screenshots and project evidence demonstrate the core workflow.

19. References and Project Evidence

- VITyarthi project assignment instructions supplied for the project review.
- Campus Bus Pass Management System source project.
- TERMINAL OUTPUT.pdf containing execution evidence.

Appendix A – Sample Console Workflow

=== CAMPUS BUS PASS SYSTEM ===

1. Add student
2. Add route
3. Generate pass
4. Verify pass
0. Exit

Choose: 1

Registration number: XPA

Name: PARAS

Category: 1

Phone: 7979

Student added.

Choose: 2

Route name: COLLEGE

Source: BUS

Destination: SATNA

Fee: 1000

Route added with ID: 5

Choose: 3

Registration number: XPA

Route ID: 5

PASS GENERATED SUCCESSFULLY

Pass ID: BP-2026-00003

Registration No: XPA

Route ID: 5

Valid From: 2026-09-13

Valid To: 2026-10-13

Status: ACTIVE

Choose: 4

Pass ID: BP-2026-00001

VALID: OK

Appendix B – Final Submission Checklist

- ☐ GitHub repository created and project source files pushed.
- ☐ README.md included at the repository root.
- ☐ screenshots/ directory contains the five project screenshots.
- ☐ TERMINAL OUTPUT/ contains the execution evidence PDF.
- ☐ Compiled output is excluded from source control where appropriate.

- ☐ campus_bus_passes.dat is kept as a local ignored runtime data file.
- ☐ Project report is prepared for review/submission according to the course instructions.

Design and UML Diagrams

The following diagrams document the implemented Campus Bus Pass Management System. They reflect the current console workflow, service/DAO structure, domain classes, and local serialized storage used by the application.

Use Case Diagram

1. Use Case Diagram — Campus Bus Pass Management System

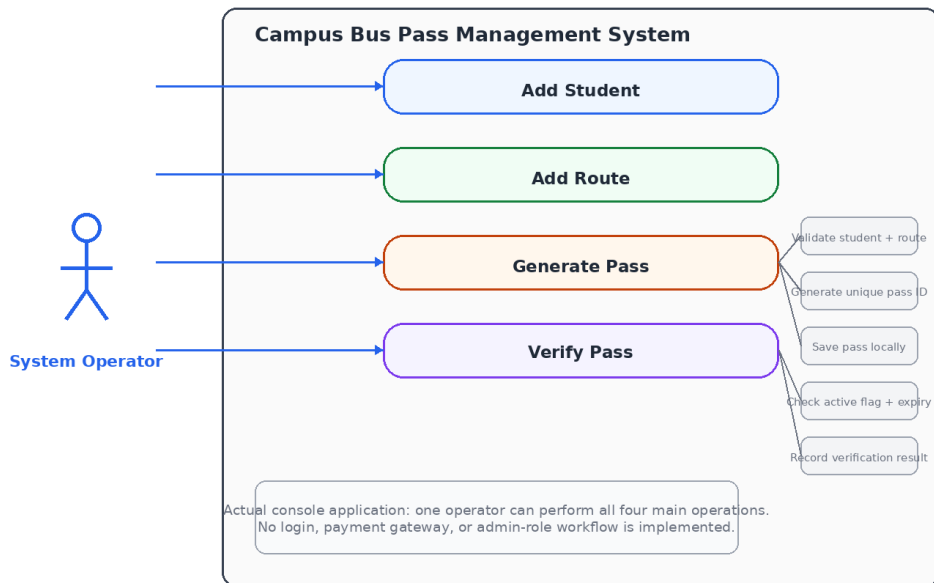


Figure 6. Use Case Diagram

Workflow Diagram

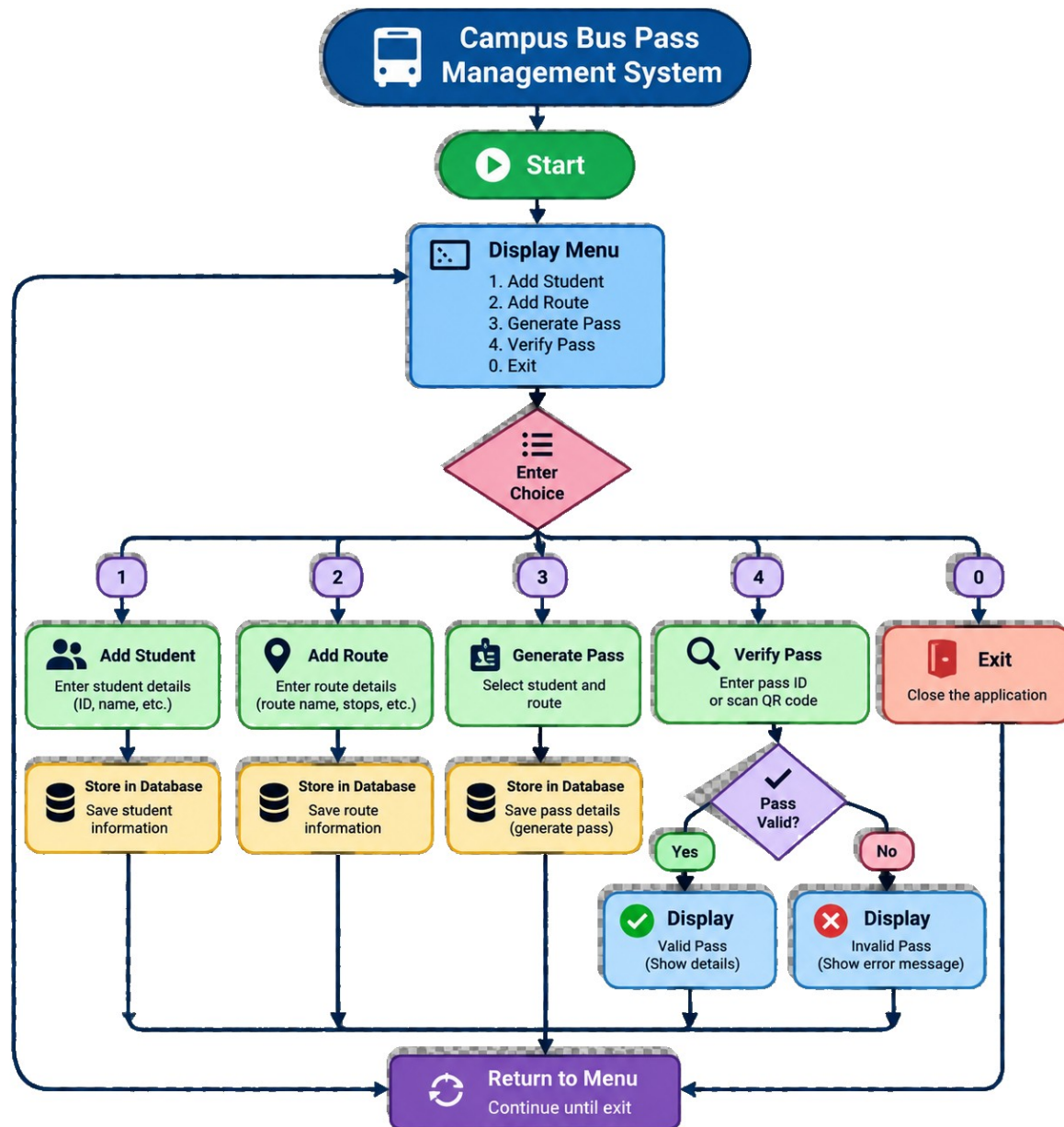


Figure 7. Workflow Diagram

Sequence Diagram

2. Sequence Diagram — Generate Bus Pass

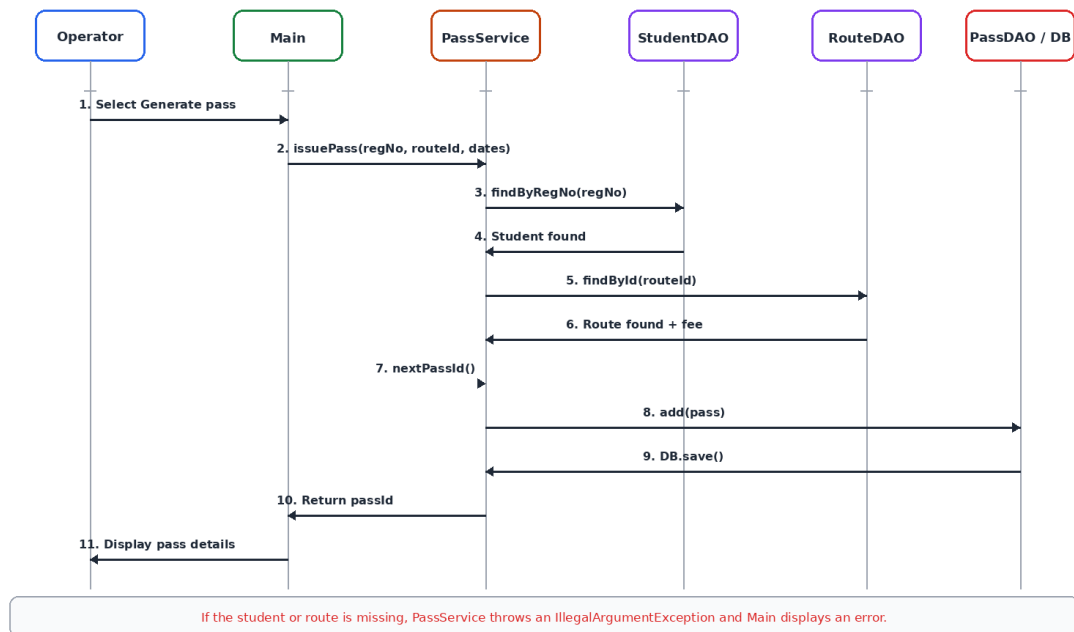


Figure 8. Sequence Diagram — Generate Bus Pass

Class Diagram

3. Class Diagram — Core Java Classes

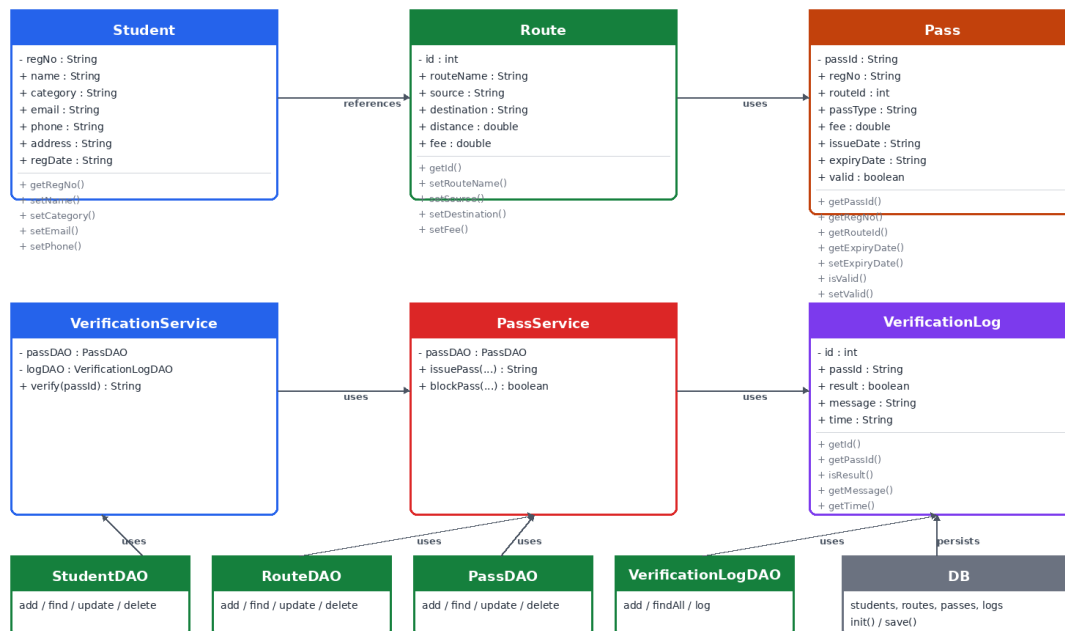


Figure 9. Class Diagram

ER / Storage Diagram

4. ER / Storage Diagram — Local File-Based Persistence

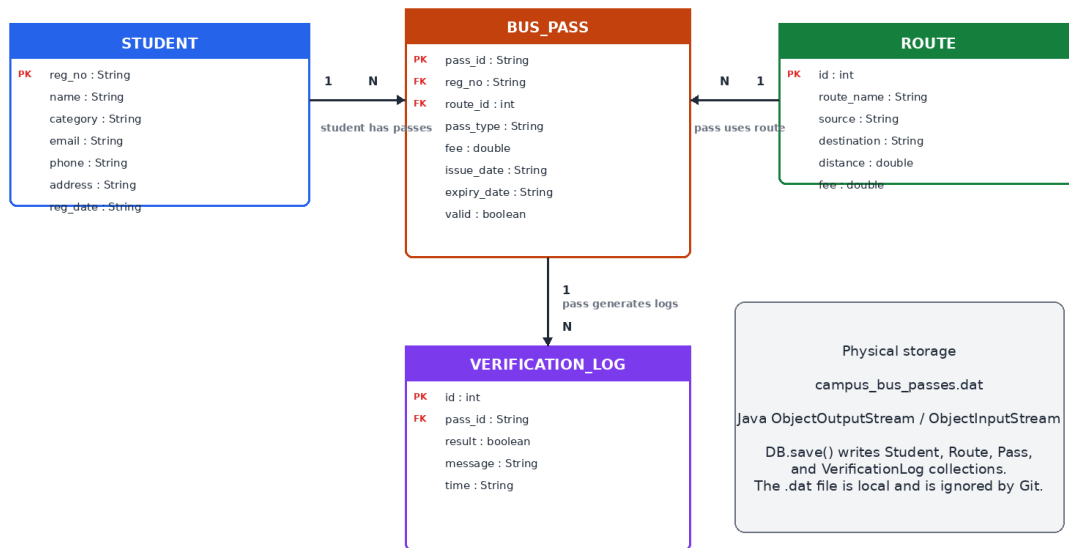


Figure 10. ER / Storage Diagram

Design Decisions and Rationale

Layered organization: The application separates menu coordination, model objects, DAO persistence, service logic, database/storage, and input handling. This keeps responsibilities focused and makes the project easier to maintain.

DAO-based persistence access: StudentDAO, RouteDAO, PassDAO, and VerificationLogDAO isolate storage operations from business rules. The service layer can therefore perform validation and workflow logic without directly manipulating the storage implementation.

Local object persistence: The current implementation persists Student, Route, Pass, and VerificationLog collections in the local campus_bus_passes.dat file using Java object serialization. This keeps the project lightweight and dependency-free for a console-based academic application.

Validation before pass issuance: PassService verifies that the student and route exist, checks the date range, obtains the configured route fee, generates a unique pass ID, creates the Pass object, and saves it.

Verification logging: VerificationService checks pass existence, active validity, and expiry. Each verification outcome is recorded through VerificationLogDAO, providing traceability for checks.

Console-first interface: A numbered console menu was selected to focus the implementation on Java fundamentals, object-oriented programming, modularity, validation, and persistence rather than GUI development.

Challenges Faced

- Keeping menu handling separate from business logic and persistence responsibilities.
- Validating student and route references before issuing a pass.
- Generating unique pass IDs and assigning monthly validity dates consistently.

- Persisting multiple object collections between program runs using local serialized storage.
- Handling valid, blocked, expired, and missing-pass verification outcomes while recording verification logs.
- Preparing a clear project structure and documentation suitable for academic evaluation.

Final Design Evidence

Together, the five diagrams document the system from user interaction through workflow, runtime sequence, class responsibilities, and persistent data relationships. The diagrams should be read together with the source code and execution screenshots included in this report.

Final Submission Checklist

- GitHub repository contains the Java source code and modular package structure.
- README.md is present at the repository root.
- statement.md is present at the repository root.
- Five execution screenshots are included in screenshots/.
- Five design diagrams are included in report/diagrams/.
- Local campus_bus_passes.dat runtime data is excluded from Git tracking.
- Final report includes requirements, architecture, diagrams, implementation, testing, screenshots, challenges, and future enhancements.
- Final PDF is prepared for portal submission.